

Pool AI - 14.1 Continuous

A reinforcement-learning pool player. Demo writeup, 2026-05-23.

Play the demo

Just open this URL in any browser (phone, tablet, laptop):

<https://all-newsletter-gentleman-leg.trycloudflare.com>

1. What it is

A pool AI that plays Straight Pool (14.1 Continuous), trained with reinforcement learning rather than written as hand-tuned heuristics. It pockets balls, manages clusters, plans end-of-rack sequences (key ball -> break ball -> scatter the rack), and runs multiple racks in succession. First-try run on the live demo as of writing: 86 balls.

The aim was not just to be accurate at individual shots, but to learn the *game* - decide between equally pocketable shots based on what comes next, preserve break balls, set up clean continuations after reracking. Several recognizable 14.1 strategic patterns emerged without being explicitly programmed.

2. Architecture (simplified)

Three pieces. Each shot decision flows top-to-bottom:

(a) Shot enumerator - pure geometry, not learned

Given current ball positions and the cue ball, generates the list of legal direct shots (ball -> pocket pairs) via line-of-sight and ghost-ball computation. Blocking checks, cushion-corridor clearance, max cut angle (80 degrees). No network involved here - shot legality is a solved geometric problem and forcing the network to learn it wastes capacity.

(b) Transformer network - the policy and value head

- 567K parameters, embed=128, heads=8, layers=4.
- Inputs are tokens: one per ball on the table, one per pocket, one per legal-shot candidate.
- Outputs: (1) policy logits over the legal shot candidates, (2) per-shot force and spin means, (3) a scalar value estimate of the current state.
- All-CPU. Runs in single-digit milliseconds per forward pass.

(c) Depth-1 search - inference-time lookahead

At each decision, the search takes the top-K shots by policy probability, simulates each one forward in the physics engine, and re-ranks by $Q = \text{immediate_reward} + \gamma * V(\text{next_state})$. $K=4$ in production. This is what unlocks the strategic depth - the bare policy plays okay, but search is what surfaces 'pocket this ball AND clear a blocker AND leave the cue here' as preferable to 'pocket the easiest shot.' About 5-40 ms per decision.

Data flow

state (balls + cue)

```

    |
    v
shot_enumerator -> list of legal shots
    |
    v
transformer -> policy probs, force/spin per shot, V(state)
    |
    v
search: for top-K shots, run physics rollout,
      score by imm_reward + gamma * V(next)
    |
    v
best-Q (shot, force, spin)
    |
    v
physics engine executes the shot
```

3. How it was trained

Training was a long sequence of incremental versions, each addressing a specific weakness observed in the previous one. Two phases of training method, both warm-starting from the previous best checkpoint:

- Phases 1-3 (early): Standard PPO on simpler sub-tasks - aim, then pocket straight-in, then cut angles. Built up the basic shot mechanics.
- Phase 7 (current): Token-based transformer trained with PPO, then AlphaZero-style distillation - run search during training, use the search-chosen action as the supervised target for the policy. This closed most of the gap between 'bare policy' and 'policy + search' play.

Reward shaping (additive)

- +10 per ball pocketed (pocket_reward).
- +5 (eor_bonus) for preserving the better break-ball candidate at n=2 balls remaining; -5 for pocketing it instead. Half-strength at n=3.
- +10 (post-rerack scatter bonus) if the break shot scatters 3+ balls.
- +1.5 (next_shape_bonus_max) scaled by the ease of the easiest legal next shot - 'leave good shape.'
- -(up to 6) shape_bonus_max penalty proportional to current shot's difficulty (cut angle + total distance). Discourages 78-degree-cut heroics when easier shots exist.
- +1 per ball pocketed that was within 3 inches of a cushion (rail_shot_bonus) - counters learned aversion to rail shots.
- Small penalties for cue-ball travel distance, multi-ball ricochets (encourages clean isolation).

Curriculum drills (env mix)

About 40 percent of training episodes are short curriculum drills instead of full racks:

- Rail-frozen drill: single object ball within 1.5 inches of a rail - trains rail-shot mechanics.
- 3-ball end-of-rack drill: key1 + key2 + break-ball - trains the EOR sequencing pattern.
- Key-and-break drill: classic curriculum scenario for the rack-opening break.

- Rail-ball break drill (newest, in v23): 14 reracked balls + 1 break ball near a rail + cue ready for the break - targets a specific failure mode where the network uses weak force when the break ball is on a rail.

4. Strategies that emerged without being told

These behaviors arose from the combination of long-horizon rewards (run length, EOR, post-break scatter) and search at inference, not from specific reward shaping for the behavior:

- Side-pocket break setup. Network leaves the break ball mid-table when possible, then pockets it into a side pocket while sending the cue ball into the rack apex with calibrated force. Classic Mosconi-era pattern.
- Blocker removal. Picks lower-probability shots that simultaneously pocket a ball AND clear a blocker from a pocket throat, when the long-horizon Q says it's worth it.
- Cluster nudging. When two balls are touching, sometimes chooses a lower-probability shot that brushes them apart while pocketing a different ball - 'deal with the cluster now before it bites later.'
- Multi-rack continuation. Reliably executes key-ball -> break-ball -> rack-scatter -> continue on the new rack, often across 5+ reracks.

Caveat: most of these require search ON to reliably surface. The value head has learned the depth; the bare policy doesn't always expose it. Distillation has narrowed but not closed this gap.

5. How to play the demo

Open the URL on the cover page. No login, no install. Layout works best on a laptop or tablet in landscape; phone portrait is cramped.

Controls

Start / Pause / Reset

Start a game, pause animation, or reset to a fresh rack.

Auto vs Manual

Auto: AI plays through the whole game on its own. Manual: AI shows you the planned shot, you click Shoot to execute (also lets you inspect the per-shot probability annotations).

Auto delay slider

Sets the pause between shots in Auto mode. 0 = as fast as the animation runs. Useful for watching long runs without 5 seconds per shot.

Trajectory line

Shows the planned cue ball path before the animation plays. In Manual mode you also see thin dashed lines for each candidate shot with (probability | cut angle) annotations.

What's interesting to watch for

- End of rack: when 2-3 balls remain, watch the choice of which ball to pocket. The model picks the one that's NOT the preferred break ball (better line to the rack apex, in the sweet-spot distance range).
- Side pockets near end of rack: the side-pocket break pattern.

- After the break: how well the cue ball lands on the new rack of 14 - whether it has a follow-up shot ready.
- Cluster handling: if two balls are touching, watch how the network deals with them.
- Rail shots: the model is now decent on these but the 'easy ones' are still occasionally misplayed. Hard rail shots almost always pocket cleanly.

6. Known weaknesses

- Easy rail shots occasionally missed. Hard ones are reliable. Diagnosis: deterministic physics gives loose force/spin tolerances on easy shots, so the model learns a vague default that fails on certain rail geometries. An attempted fix via training noise (v21) traded EOR depth for rail calibration; further work in flight.
- Weak force on rail-ball break shots. When the post-rerack break ball is positioned near a rail, the model uses ~80 force instead of the ~240 needed to scatter the rack. Its learned rail-shot prior ('low force = safe pocket') overrides the scatter bonus. Targeted curriculum drill added in v23 (in training as of writing).
- Search-dependent strategic depth. Bare-policy play is noticeably weaker than policy+search play. More distillation would close this further.
- 78-degree-cut shots still get non-trivial probability in some positions, though they're now rarely *chosen* thanks to the search probability threshold.

7. Tech stack

- Physics: custom C simulator (pool_sim.c, compiled to libpool_sim.so), called from Python via ctypes. Calibrated friction/restitution, continuous spin factor.
- ML: PyTorch, CPU-only. Transformer policy/value net is small (567K params) and was trained over ~3-4 weeks of incremental experiments.
- Demo: a Python HTTP server (~700 LOC) serves a single static HTML page that renders the table in a canvas and posts shot decisions to the server.
- Tunnel for remote access: cloudflared quick tunnel - one command, no account.

8. The link

Play it:

<https://all-newsletter-gentleman-leg.trycloudflare.com>

The tunnel needs the host machine to keep cloudflared running. If the link stops working it's because that process was stopped - the AI itself is fine, it's just unreachable from the public internet. Direct LAN access on the same WiFi as the host is via <http://192.168.1.90:8001/>.